

705.641.81: Natural Language Processing Self-Supervised Models

Homework 2: Language modeling + count-based LMs
+ preliminaries to neural networks

For homework deadline and collaboration policy, please see our Canvas page.

Name: **Rajiv Khandpur**

Collaborators, if any: _____

Sources used for your homework, if any: _____

Homework goals: After completing this homework, you should be able to be comfortable with various foundational concepts related to our course. Specifically, we hope you will gain the followings:

- Get more comfortable with linear algebra, specifically gradients and Jacobians!
- Gain a better grasp of count-based language models!
- Get comfortable with activation functions such as the Softmax function!
- Know how to develop your own classifier!

Concepts, intuitions and big picture

1. The n -gram models we've shown make the n -th order Markov assumption, i.e. that the distribution of words depends only on the previous $n - 1$ words. What properties of language will it not capture? Discuss (briefly) several distinct ways in which this assumption is false for natural language.
 - a. The n -gram models are generally unable to capture *long distance* dependencies in natural language. For instance in the sentence "The computer which they had just put into the multipurpose room on the twelfth floor crashed." *computer* and *crashed* are linked but only an n -gram model with a large window would pick up both words. This leads to the second problem with n -gram models, *sparsity*.
 - b. With large values of n , n -gram models run into the sparsity problem where most of the n -grams do not appear in the training data and therefore have zero counts.
 - c. With a shorter window, these models then run into semantics and coherence problems. With small values of n we are likely to string together n -grams that don't make any sense.
2. Follow-up to previous question: Despite the Markov assumption, n -gram models are remarkably good at predicting the next word. Discuss why this might be. What information is in the previous word(s) that makes these models perform so surprisingly well? In particular, what kinds of grammatical information do they capture?
 - a. The main reason these models perform well at predicting the next word is because in natural language the next word is typically dependent on the preceding words. For instance the word "to" usually precedes a verb or a noun and adjectives usually precede a noun. Knowing the first n -words of a sequence provides a lot of grammatical clues to help predict the next word.
 - b. Another reason they appear to perform so well is due to the quality of the training data. Assuming the data is from a source with correct grammar (like news articles instead of a twitter feed) the frequency of grammatically correct n -grams in the training data will mean the model's ability to make grammatically correct predictions will be improved.
3. Explain how are perplexity and cross-entropy loss related?
 - a. Perplexity is the inverse probability of an n -gram normalized by the number of words in the n -gram. Based on this we can express perplexity as $ppl(w_1, \dots, w_n) = P(w_1, \dots, w_n)^{-1/n}$. We can rewrite this as logits

(log probabilities) as $ppl(w_1, \dots, w_n) = 2^H$ where

$H = \frac{-1}{n} \sum \log_2 P(w_i | w_1, \dots, w_{i-1})$. Here H starts to look a lot like the cross-entropy between two distributions where

$H(p, q) = - \sum p(x) \log(q(x))$. So we can interpret perplexity as the cross-entropy loss exponentiated.

4. For a vocabulary of $|V|$ words, what would you expect perplexity to be if your model predictions were completely random? Compute the corresponding cross-entropy loss for $|V| = 2000$ and $|V| = 10000$, and keep this in mind as a baseline.
 - a. As discussed in the lectures, perplexity is bounded between 1.0 (model is perfect and makes prediction with complete certainty) and $|V|$ (model assigns equal probability to all words). If the model's predictions are completely random, they must be applying a uniform probability over the vocabulary so the perplexity of such a model would be $|V|$.
 - b. Because we have
$$\forall w \in V : P(w | w_{1:i-1}) = \frac{1}{|V|} \implies ppl(D) = 2^{-\frac{1}{n} n \log_2(\frac{1}{|V|})}$$
We can work out that the exponent simplifies to just $\log_2(|V|)$.
$$2^{-\log_2(\frac{1}{|V|})} = 2^{-(\log_2(1) - \log_2(|V|))} = 2^{-(0 - \log_2(|V|))} = 2^{\log_2(|V|)}$$
Therefore the cross-entropy loss for $|V| = 2000$ is simply $\log_2(2000) \approx 10.966$
 - c. Similarly the cross-entropy loss for $|V| = 10000$ is simply $\log_2(10000) \approx 13.288$
5. Which of these are reasons for the recent wave of neural networks taking off? (check the options that apply.)
 - ☐ (check) We have access to a lot more computational power.
 - ☐ Neural Networks are a brand new field.
 - ☐ (check) We have access to a lot more data.
 - ☐ There has been significant improvements in benchmarks in speech recognition and NLP.

6. What does a neuron compute?
- A neuron computes an activation function followed by a linear function ($z = Wx + b$)
 - (check) A neuron computes a linear function ($z = Wx + b$) followed by an activation function.
 - A neuron computes a function g that scales the input x linearly ($Wx + b$).
 - A neuron computes the mean of all features before applying the output to an activation function.
7. What is the point of applying a Softmax function to the logits output by a sequence classification model?
- It softens the logits so that they're more reliable.
 - It applies a lower and upper bound so that they're understandable.
 - (check) The total sum of the output is then 1, resulting in a possible probabilistic interpretation.

Linear Algebra Recap

Gradients

Consider the following scalar-valued function:

$$f(x, y, z) = x^2y + \sin(z + 6y) .$$

1. Compute partial derivatives with respect to x , y and z .

a. $\frac{\partial f}{\partial x} = 2xy$

b. $\frac{\partial f}{\partial y} = x^2 + 6\cos(z + 6y)$

c. $\frac{\partial f}{\partial z} = \cos(z + 6y)$

2. We can consider f to be a function that takes a vector $\theta \in \mathbb{R}^3$ as input, where $\theta = [x, y, z]^\top$. Write the gradient as a vector and evaluate it at $\theta = [3, \pi/2, 0]^\top$.

a. $\nabla f(\theta) = [2xy, \quad x^2 + 6 \cos(z + 6y), \quad \cos(z + 6y)]^T$

b. $\nabla f(\theta) \Big|_{\theta=[3, \pi/2, 0]^T} = [3\pi, \quad 3, \quad -1]$

Gradients of vectors

Let $\mathbf{x}, \mathbf{c} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{n \times n}$. For the following parts, before taking any derivatives, identify what the derivative looks like (is it a scalar, vector, or matrix?) and how we calculate each term in the derivative. Then carefully solve for an arbitrary entry of the derivative, then stack/arrange all of them to get the final result.

- Show that $\frac{\partial}{\partial \mathbf{x}} (\mathbf{x}^\top \mathbf{c}) = \mathbf{c}^\top$.
 - o The result of $\vec{x}^T \vec{c}$ is the dot product of the two vectors and is a scalar. So we must differentiate a scalar with respect to the vector $\vec{x}$ so the result must be a vector.
 - o Since $\vec{x}^T \vec{c} = x_1 c_1 + \dots + x_n c_n$ the derivative with respect to $\vec{x}$ is $\left[\frac{\partial}{\partial x_1} x_1 c_1, \quad \dots, \quad \frac{\partial}{\partial x_1} x_n c_n \right] = [c_1, \quad \dots, \quad c_n] = \vec{c}^T$
- Show that $\frac{\partial}{\partial \mathbf{x}} \left(\|\mathbf{x}\|_2^2 \right) = 2\mathbf{x}^\top$.
 - o $\|\vec{x}\|_2^2 = \vec{x}^T \vec{x} = \sum x_i x_i = \sum x_i^2$ which is a scalar. So we are looking to differentiate a scalar with respect to the vector $\vec{x}$ so the result must be a vector.
 - o To compute the individual entries by differentiating each element of the vector

$$\frac{\partial}{\partial \vec{x}} (\|\vec{x}\|_2^2) = \left[\frac{\partial}{\partial x_1} x_1^2, \quad \dots, \quad \frac{\partial}{\partial x_n} x_n^2 \right] = [2x_1, \quad \dots, \quad 2x_n] = 2[x_1, \quad \dots, \quad x_n] = 2\vec{x}^T$$

- Show that $\frac{\partial}{\partial \mathbf{x}}(\mathbf{A}\mathbf{x}) = \mathbf{A}$.
 - The expression $\mathbf{A}\vec{x}$ is a matrix transformation of a vector resulting in a vector and we can express the individual elements of the transformed vector as $(\mathbf{A}\vec{x})_i = \sum A_{ij}x_j$. We are attempting to differentiate a vector-valued function with an output in $\mathbb{R}^n$ with respect to a vector in $\mathbb{R}^n$. The result of this is a Jacobian matrix in $\mathbb{R}^{n \times n}$.
 - We can treat the individual entries of the output of $\mathbf{A}\vec{x}$ as a vector of dot products $[\vec{A}_1\vec{x}, \dots, \vec{A}_n\vec{x}]$ and then differentiate that vector with respect to $\vec{x}$ gives us $\frac{\partial}{\partial \vec{x}} [\vec{A}_1\vec{x}, \dots, \vec{A}_n\vec{x}] = [\vec{A}_1, \dots, \vec{A}_n]$ which is a matrix of the columns of the original matrix $\mathbf{A}$.
- Show that $\frac{\partial}{\partial \mathbf{x}}(\mathbf{x}^T \mathbf{A} \mathbf{x}) = \mathbf{x}^T (\mathbf{A} + \mathbf{A}^T)$.
 - The output of the expression $\vec{x}^T \mathbf{A} \vec{x}$ is a vector dotted with the linear transformation of that same vector which is a scalar. We can rewrite the expression as $\vec{x}^T \mathbf{A} \vec{x} = \sum_{j=1}^n \sum_{i=1}^n x_j A_{ji} x_i$ which is a scalar. So we are looking to differentiate a scalar with respect to a vector. Similar to the first problem, the result of this is a vector.
 - To differentiate this expression, it helps to express it as a nested function so that we can apply the chain rule. We can rewrite the expression as $f(\vec{x}) = g(\vec{x}, \vec{u})$ where $g(\vec{x}, \vec{u}) = \vec{x}^T \vec{u}$ and $\vec{u} = u(\vec{x}) = \mathbf{A}\vec{x}$. Taking the derivative of this we get $\frac{\partial}{\partial \vec{x}}(g(\vec{x}, \vec{u})) = \frac{\partial}{\partial \vec{x}}(\vec{x}^T u(\vec{x}))$. We can apply the product rule first and then the chain rule to get $\frac{\partial}{\partial x}(x^T u(x)) = u(x)^T + x^T \frac{\partial u(x)}{\partial x} = u(x)^T + x^T A$. We can then rewrite this as $= (Ax)^T + x^T A = x^T A^T + x^T A = x^T (A + A^T)$.

Note:

The above equations follow the [numerator layout](#) notation which is commonly following in linear algebra, but there are alternative notational conventions as well.

Jacobian

Consider the following vector function from $\mathbb{R}^3$ to $\mathbb{R}^3$:

$$\mathbf{f}(\boldsymbol{\theta} = [x_1, x_2, x_3]) = \begin{cases} \sin(x_1 x_2 x_3) \\ \cos(x_2 + x_3) \\ \exp\left(-\frac{1}{2}x_3^2\right) \end{cases}$$

1. What is the Jacobian matrix¹ of $\mathbf{f}(\boldsymbol{\theta})$?

- a. For each i-th row and j-th column in the Jacobian, the entry will be the derivative of the i-th function with respect to the j-th variable. Since each of the functions is a nested function we can apply the chain rule.

$$\begin{aligned} \text{b. } \nabla f(\theta) = J(\theta) &= \begin{bmatrix} \frac{\partial}{\partial x_1}(\sin(x_1 x_2 x_3)) & \frac{\partial}{\partial x_2}(\sin(x_1 x_2 x_3)) & \frac{\partial}{\partial x_3}(\sin(x_1 x_2 x_3)) \\ \frac{\partial}{\partial x_1}(\cos(x_2 + x_3)) & \frac{\partial}{\partial x_2}(\cos(x_2 + x_3)) & \frac{\partial}{\partial x_3}(\cos(x_2 + x_3)) \\ \frac{\partial}{\partial x_1}(\exp(-\frac{1}{2}x_3^2)) & \frac{\partial}{\partial x_2}(\exp(-\frac{1}{2}x_3^2)) & \frac{\partial}{\partial x_3}(\exp(-\frac{1}{2}x_3^2)) \end{bmatrix} \\ &= \begin{bmatrix} x_2 x_3 \cos(x_1 x_2 x_3) & x_1 x_3 \cos(x_1 x_2 x_3) & x_1 x_2 \cos(x_1 x_2 x_3) \\ 0 & -\sin(x_2 + x_3) & -\sin(x_2 + x_3) \\ 0 & 0 & -x_3 \exp(-\frac{1}{2}x_3^2) \end{bmatrix} \end{aligned}$$

2. Evaluate the Jacobian matrix of $\mathbf{f}(\boldsymbol{\theta})$ at $\boldsymbol{\theta} = [1, \pi, 0]$.

$$\text{a. } J(\theta = [1, \pi, 0]) = \begin{bmatrix} 0 & 0 & \pi \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

¹ https://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant

n-gram Language Models

A toy *n*-gram language model

Consider the following vocabulary,

$V = \{BOS, EOS, who, framed, roger, rabbit, the\}$ where BOS is the dummy token indicating the beginning of a sentence, and EOS indicates the end of a sentence.

Consider the following training data:

BOS who EOS

BOS who framed roger EOS

BOS roger rabbit EOS

BOS who framed roger rabbit EOS

BOS roger framed who EOS

BOS who framed the rabbit EOS

1. Compute the following probabilities: $P(\text{rabbit})$, $P(\text{rabbit} | \text{roger})$, $P(\text{EOS} | \text{rabbit})$.
 - a. I wrote a little code in a “scrap” Jupyter notebook in the repo to help check and calculate the unigram and bigram probabilities.
 - i. Treating the *BOS* and *EOS* tokens as part of the vocabulary I get $P(\text{rabbit}) = 3/29 = 0.1034$
 - ii. Still treating the *BOS* and *EOS* tokens as part of the vocabulary I get $P(\text{rabbit} | \text{roger}) = 0.5$ since there are 4 occurrences of ‘roger’ and following 2 of those is the word ‘rabbit’.
 - iii. $P(\text{EOS} | \text{rabbit}) = 1.0$ since the only times the word ‘rabbit’ appears is at the end of the sentence.
2. Briefly explain what the sparsity problem is in *n*-gram language models?
 - a. The sparsity problem is that many coherent and grammatically valid word sequences in the vocabulary won’t appear in the training data even with a large corpus and will end up with zero counts for those sequences. A good example of this was provided in the lectures where the Shakespeare corpus had a relatively large vocabulary of 29k words with 844mm possible bigrams but only 300k bigrams were actually observed in the corpus. This means that more than 99% of the possible bigrams would have zero counts and many of those bigrams would be coherent and grammatically correct word sequences.

Challenges of linear classifiers of sentences

A simple way to compute a representation for a phrase s is to add up the representations of the words in that phrase: $\text{repr}(s) = \sum_{w \in s} v_w$, where $w \in s$ are the word in s and $v_w \in \mathbb{R}^d$ is the embedding for word w .

1. Now, consider sentiment analysis on a phrase in which the predicted sentiments are

$$f(s; \theta) = \theta \cdot \text{repr}(s),$$

for some choice of parameters θ . Note that here $\theta \in \mathbb{R}^d$ and “ $\cdot$ ” is the “dot-product”. Prove that in such a model, the following inequality cannot hold for any choice of θ and word embeddings:

$$\begin{aligned} f(\text{good}; \theta) &> f(\text{not good}; \theta) \\ f(\text{bad}; \theta) &< f(\text{not bad}; \theta) \end{aligned}$$

Thereby, showing the inadequacy of this model in capturing negations

- A. Assuming that ‘good’, ‘not good’, ‘bad’, and ‘not bad’ are the actual phrases being fed into the model, we can solve this algebraically. Since the output of the model is just the dot product of the weights θ by the sum of the embeddings of the words in the phrase we can use the distributive property to simplify the inequalities:

- $f(\text{good}; \theta) > f(\text{not good}; \theta)$ is equiv to $\theta \cdot v_{\text{good}} > \theta \cdot (v_{\text{not}} + v_{\text{good}})$
distributing θ on the right side we get $\theta \cdot v_{\text{good}} > \theta \cdot v_{\text{not}} + \theta \cdot v_{\text{good}}$.
Subtracting $\theta \cdot v_{\text{good}}$ from both sides gives us $0 > \theta \cdot v_{\text{not}}$.
- We can express the other inequality using the same algebraic tools.
 $f(\text{bad}; \theta) < f(\text{not bad}; \theta)$ is equiv to $\theta \cdot v_{\text{bad}} < \theta \cdot v_{\text{not}} + \theta \cdot v_{\text{bad}}$. We can then subtract $\theta \cdot v_{\text{bad}}$ from both sides to get $0 < \theta \cdot v_{\text{not}}$ which is a contradiction to the evaluation of the first inequality and therefore the inequalities cannot hold.

2. **Extra Credit:** Construct another example of a pair of inequalities similar to the ones above that cannot both hold.
 - a. A pair of similar inequalities resulting in a similar contradictions would be:
 - i. $f(\text{good}; \theta) < f(\text{very good}; \theta)$ and $f(\text{bad}; \theta) > f(\text{very bad}; \theta)$
 - ii. Contradiction $\theta \cdot v_{\text{very}} < 0$ and $\theta \cdot v_{\text{very}} > 0$

3. **Extra Credit:** Consider a slight modification to the previous predictive model:

$$f(s; \theta) = \theta \cdot \text{ReLU}(\text{repr}(s)),$$

where ReLU (rectified linear function) is defined as:

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0. \end{cases}$$

Given this choice of predictive function, show that it is possible to satisfy the above inequalities for some choice of θ . **Hint:** Show there exists parameters θ and word embeddings v_{good} , v_{bad} and v_{not} that the inequalities are satisfied.

- The algebra that proved the previous inequalities were impossible is no longer valid since we are applying the ReLU function to the embeddings before dotting them with the weights and we no longer have a proof that it's impossible. I think the solution is somewhat simple to find embeddings and weights that would satisfy the inequalities but I'm running out of time to complete the homework so I may not get to finish this extra credit.
4. Given the above result, explain (in 1-2 sentence) why the use of neural networks (which have more complexity than linear models)
- a. I interpreted this question as asking us to explain why neural networks are better than linear models. This is because linear models can only ever produce a combination and linear transformation of features by fixed weights. Neural networks add nonlinearity through functions like ReLU or the sigmoid function that break the limitation that we demonstrated through the inequalities.

Softmax function

Remember the Softmax functions from the class:

Softmax: $\sigma(\mathbf{z}) \triangleq [\sigma(\mathbf{z})_1, \dots, \sigma(\mathbf{z})_K]$ s.t. $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$ for $i = 1, \dots, K$ and $\mathbf{z} = (z_1, \dots, z_K)$

1. Prove that Softmax is invariant to constant offsets in the input, i.e., for any input vector $\mathbf{z}$ and any constant c ,

$$\sigma(\mathbf{z}) = \sigma(\mathbf{z} + c)$$

Pro tip: We make use of this property in practice to increase the numerical stability of our models. Specifically, using $c = -\max_{i \in \{1 \dots K\}} z_i$, i.e. subtracting its maximum element from all elements of $\mathbf{z}$ would prevent numerical instability due to large values.

- For each element in $\vec{z}$ we have $\sigma(z + c)_i = \frac{e^{z_i+c}}{\sum e^{z_j+c}} = \frac{e^{z_i} \cdot e^c}{\sum e^{z_j} \cdot e^c}$. We can then factor out the common e^c factors to get $\frac{e^c}{e^c} \frac{e^{z_i}}{\sum e^{z_j}} = \frac{e^{z_i}}{\sum e^{z_j}} = \sigma(z)_i$. This holds for every element in $\vec{z}$ so the full vectors are equal for any constant c .

2. Softmax maintains the relative order of the elements in $\mathbf{z}$. In particular, show that the largest index is intact after applying Softmax:

$$\arg \max_{i \in \{1 \dots K\}} z_i = \arg \max_{i \in \{1 \dots K\}} \sigma(\mathbf{z})_i$$

- For each element in $\sigma(\vec{z})$ the denominator is the same fixed positive number $\sum e^{z_i}$. If we look at any two indices i, j such that $z_i > z_j$, we can examine just the numerators after the application of the Softmax function since they would share the same denominator.
- The exponentiation function e^x is strictly monotonically increasing so if $a > b$ then $e^a > e^b$.

- Therefore

$$z_i > z_j \implies e^{z_i} > e^{z_j} \implies \frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} > \frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}} \implies \sigma(z)_i > \sigma(z)_j$$

- Therefore $\arg \max z_i = \arg \max \sigma(\vec{z})_i$

3. Define the Sigmoid function as follows:

$$\text{Sigmoid: } S(x) \triangleq \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} = 1 - S(-x).$$

Prove that Softmax is equivalent to the Sigmoid functions when the number of possible labels is two: $K = 2$. Specifically:

$$\sigma_1([z_1, z_2]) = S(z_1 - z_2)$$

- Since we have $K = 2$ we can let $\mathbf{z} = (z_1, z_2) \in \mathbb{R}^2$

- Applying softmax we have $\sigma(\mathbf{z})_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}$

- Multiplying both the numerator and the denominator by $\frac{1}{e^{z_1}}$ we get $\sigma(\mathbf{z})_1 = \frac{1}{1 + \frac{e^{z_2}}{e^{z_1}}} = \frac{1}{1 + e^{z_2 - z_1}} = \frac{1}{1 + e^{-(z_1 - z_2)}}$.

- We can see that this is the sigmoid function evaluated at $x = z_1 - z_2$

- Taking

4. **Extra Credit:** Next, let's extend (1) to prove the following inequality:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \geq \prod_{\substack{j=1 \\ j \neq i}}^K \frac{1}{1 + e^{-(z_i - z_j)}} = \prod_{\substack{j=1 \\ j \neq i}}^K S(z_i - z_j)$$

Hint: Use the following inequality $\left(1 + \sum_i \alpha_i\right) \leq \prod_i (1 + \alpha_i)$ where each $\alpha_i \geq 0$.

- Unfortunately didn't have time to attempt this one.

Programming

In this programming homework, we will

- build a simple count-based (n-gram) language model.
- implement your own gradient descent optimization for the classifier you built in homework 1.

Skeleton Code and Structure:

The code base for this homework can be found at [this GitHub repo](#) under the `hw2` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding `.py` files. The code base has the following structure:

- `ngram_lm.py` implements a n-gram language model on a subset of Wikipedia.
- `gradient_descent.py` reuse the sentiment classifier on movie reviews you implemented in homework 1, with additional requirements to implement manual softmax, cross-entropy loss, and gradient updates.
- `main.py` provides the entry point to run your implementations in both `ngram_lm.py` and `backprop.py`.
- `hw2.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.

TODOs (Copy from your HW1). We are reusing most of the `model.py` from homework 1 as the starting point for the `gradient.py` - you will see in the skeleton that they look very similar. Moreover, in order to make the skeleton complete, for all the `# TODO` (**Copy from your HW1**), please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in homework 1.)

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file

Count-based LMs

In the lecture, we have learned language modeling as building probabilistic distribution (marginal and joint) over language. Moreover, we can estimate such distributions (e.g. $P(X_t | X_1, \dots, X_{t-1})$) by *counting*. For example

$$P(\text{mat} | \text{the cat sat on the}) \approx \frac{\text{count}(\text{"the cat sat on the mat"})}{\text{count}(\text{"the cat sat on the"})}$$

We are going to implement such a count-based LM, i.e. n-gram language model.

Data Loading

We will use (a subset of) the [WikiText](#) dataset for building the n-gram LM. The WikiText language modeling dataset is a collection of over 100 million tokens extracted from the set of verified Good and Featured articles on Wikipedia. Spend a few minutes reading a few examples on Huggingface to get a better sense of what this dataset looks like. We will use Huggingface's datasets library to download this dataset locally. For efficiency, we only use the first $1e5$ training samples to create our LM.

Read the `load_data` function in `ngram_lm.py` for how we obtain the data.

Preprocessing: Sentence Split and Tokenization

As you have seen, each data sample consists of a Wikipedia paragraph. To perform our n-gram counting, we need first split each paragraph into sentences ² and then each sentence into a sequence of tokens (i.e. *tokenization* that we introduced in homework 1). In particular, we will use [an algorithm](#) by our very own Philipp Koehn for sentence splitting; and use a *sub-word tokenizer* for tokenization. For now, we are using an existing sub-word tokenizer of an existing model from the Huggingface library, but in the future, we will also build our own tokenizer.

A *Sub-word tokenizer* which, as it should be clear from the name, splits each sentence into units smaller than words, based on their frequency. For example, the word "Potentials" is broken into four sub-words: 'Po', '##ten', '##tial', '##s', where '##' is a special symbol indicating that the sub-word is in the middle of a word.

Sub-word tokenization might initially seem like a bad idea since we are breaking up the word. But it brings it several benefits.

- Handling Out-of-Vocabulary (OOV) words: Sub-word tokenization allows the representation of rare or unseen words as a combination of sub-words or tokens that have been seen in the training data, thus reducing the OOV problem.

² we count n-gram status within sentence boundary

- Improved vocabulary size: It allows for a more compact vocabulary size, reducing the size of the language model, and increasing its efficiency.
- Better language model performance: Sub-word tokenization results in a better representation of morphologically rich languages, where words are formed by combining roots and affixes, leading to improved language model performance.
- Cross-lingual compatibility: Sub-word tokenization is language-agnostic, and models trained on sub-word tokens from one language can be applied to other languages, improving cross-lingual compatibility.

How are sub-tokenizers built? We will delve into that in a few weeks! For now, we will just use them!

Read the `sentence_split_and_tokenize_demo` for how exactly an wikitext paragraph is converted into sentences and then tokens.

Build the N-gram LM: Let's Count!

TODOs: read the `create_ngrams` function which iteratively processes each paragraph and counts the n-gram statistics, and completes the following lines under three

TODOs:

- tokenize the words in the sentence: that takes in a `sentence` and convert it into `tokens` as a list of tokens with the tokenizer.
Hint: check out how to do tokenization in `sentence_split_and_tokenize_demo`
- count n-gram statistics over each list of tokens, record them in
 - `ngrams`: the `Counter` for n-grams count
 - `ngram_context`: the `Counter` for context (i.e. (n-1)-grams) count
 - `next_word_candidates`: the `Dictionary` that keeps track of all possible candidate next tokens given the context, i.e. all tokens that, concatenated with the (n-1)-gram context, constitutes an n-gram that exists in the data.

for all the `Counter` and `Dictionary`, use `tuple` of n-gram or (n-1)-gram tokens as the keys.

Hint: also check the corresponding comments in the code for specific requirements

- computes the estimated probability of the next word (token) given the context.
Hint: use the formula in [6.1](#), and also the corresponding comments in the code

Finally, the `next_word_pred` returned from the `create_ngrams`, will record for each (n-1) gram context, the most probable next word continuation. For this homework, we will set $n = 3$ so that to build a *trigram* language model.

Visualize the N-gram distribution

TODOs: Complete the `plot_next_word_prob` function in `ngram_lm.py`, and run `run_ngram` in `main.py` to plot the top-10 most probable next token given an input context. Paste the two plots for the two given contexts (provided in `run_ngram`), and describe in 2-3 sentences your findings.

Hints:

- you can check out [this tutorial](#) about how to plot bar chart with [matplotlib](#).
- read the comments in the code carefully for more info about the input.

Generation: Sample from the LM

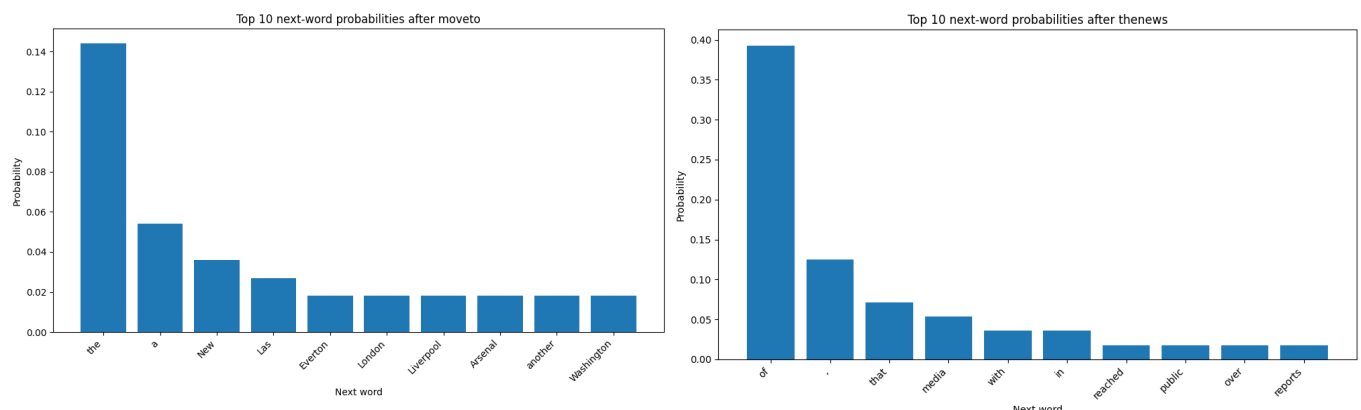
Now that we have built our LM, given any prefix³, we can sample from the LM to generate a full completion. Specifically, at each timestamp, we take the last $(n-1)$ tokens from the prefix as the context and query our LM to get the most probable next token, append it to the prefix, and continue until we sample the stop token or the maximum length is reached.

This sampling procedure is called greedy decoding, as we take the most probable next token each step, there are a few other sampling strategies like beam search, top-k, and top-p sampling. Similarly, we will get to them soon.

Read the `generate_text` function in `ngram_lm.py` for details of the generation process described above.

TODOs: Run `run_ngram` in `main.py` and paste the completion of the two given prefixes (provided in `run_ngram`), and describe in 2-3 sentences your findings.

- After the 'moveto' prefix the tokens with the highest probability are they type of words you would expect to see before the noun representing a destination ('the', 'a'). The next most probable tokens are the proper names (or beginnings of proper names) of actual places ('New' for 'New York' perhaps, 'Las' for 'Las Vegas' perhaps, 'Everton', 'London').
- After the 'thenews' prefix the token with the highest probability is 'of' which is unsurprising since that phrase is usually followed by the topic of the news. The next token is the period '.' which implies that it is often found at the end of the sentence.



Optimizing the Sentiment Classifier with (Stochastic) Gradient Descent

In the second part of this programming homework, we will revisit the sentiment classifier we built in the last homework. Instead of relying on the PyTorch built-in loss functions, gradient calculations, and weights optimization, we will delve into them and implement our own version!

Reuse Your HW1 Implementation

TODOs (Copy from your HW1): for all the `# TODO (Copy from your HW1)` in `gradient_descent.py`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in the `model.py` in homework 1).

Softmax Function

Remember the [nn.CrossEntropyLoss](#) we used in homework 1, it can be further decomposed into that 1) normalize the real-value scores of each class (e.g. the logits) into a probability distribution using softmax function, and calculate the cross entropy loss of this probability distribution against the ground truth binary distribution (In practice, PyTorch instead provides the combination of [LogSoftmax](#) with [negative log likelihood loss](#)). We will first implement the softmax function, which you have been familiar with in the lecture, and [5](#).

TODOs: Complete the `softmax` function of the `SentimentClassifier` class in `gradient_descent.py`. Note: you must implement the optimized for numerical stability version described in the **Pro tip** in [\[subsec:5.1\]](#) of [5](#).

Hint: check the comments in the code for specific input-output requirements.

A correct implementation should pass the `test_softmax` in `gradient_descent.py`.

With the softmax function, we turn our neural network into a classifier that assigns a probability distribution over the 2 sentiment classes. Specifically, denoting our input feature vector as $\mathbf{x}$, the `nn.Linear` layer transforms $\mathbf{x}$ into a **logit score** vector $\mathbf{z}$ using a weight matrix W and a bias vector $\mathbf{b}$:

$$\mathbf{z} = \mathbf{x}W + \mathbf{b}$$

This logit score has one element per class, so the weight matrix must have a size (d, c) , where c is the number of classes (output labels) and d is the number of dimensions of the input space (features). The bias vector has c elements (one per class).

The logit score is turned into probabilities using the **softmax** operator:

$$\hat{y}_j = \mathbf{P}(\text{class} = j) = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

Gradients on Cross Entropy Loss

We will start by defining an objective function that defines "goodness" for our classifier. A common choice for classification is [categorical cross-entropy loss](#) or negative log-likelihood.

A discussion or derivation of cross-entropy loss is beyond the scope of this class but a good introduction to it can be [found here](#). A discussion of what makes it superior to MSE for classification can be [found here](#). We will just focus on its properties instead.

Letting y_i denote the ground truth value of class i , and $\hat{y}_i$ be our prediction of class i , the cross-entropy loss is defined as:

$$CE(y, \hat{y}) = - \sum_i y_i \log \hat{y}_i$$

If the number of classes is 2 (which is the case here), we can expand this:

$$CE(y, \hat{y}) = - \left(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right)$$

Notice that as our probability for predicting the correct class approaches 1, the cross-entropy approaches 0. For example, if $y = 1$, then as $\hat{y} \rightarrow 1$, $CE(y, \hat{y}) \rightarrow 0$. If our probability for the correct class approaches 0 (the exact wrong prediction), e.g. if $y = 1$ and $\hat{y} \rightarrow 0$, then $CE(y, \hat{y}) \rightarrow \infty$.

This is true in the more general M -class cross-entropy loss as well,

$$CE(y, \hat{y}) = - \sum_i y_i \log \hat{y}_i, \text{ where if our prediction is very close to the true label, then}$$

the entropy loss is close to 0, whereas the more dissimilar the prediction is to the true class, the higher it is.

Practical tip: in practice, a very small ϵ is added to the log, e.g. $\log(\hat{y} + \epsilon)$ to avoid $\log 0$ which is undefined.

To optimize this objective, we will compute its gradients with respect to parameters W and b

Before doing that, let's redefine cross-entropy loss in matrix form. With a minibatch of

input features $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \cdots \\ \mathbf{x}_N \end{bmatrix} \in \mathcal{R}^{N \times d}$ where N is the number of instances per batch

(batch size), and d is the dimension of feature vectors, and each input $\mathbf{x}_j$ is a row vector

of dimension d . And the corresponding output from our network $\hat{\mathbf{Y}} = \begin{bmatrix} \hat{\mathbf{y}}_1 \\ \cdots \\ \hat{\mathbf{y}}_N \end{bmatrix} \in \mathcal{R}^{N \times c}$

and label matrix $\mathbf{Y} = \begin{bmatrix} \mathbf{y}_1 \\ \cdots \\ \mathbf{y}_N \end{bmatrix} \in \mathcal{R}^{N \times c}$, with row vectors $\mathbf{y}_j \in \{0, 1\}^c$ as one-hot

encoding of the class label, and $\hat{\mathbf{y}}_j \in [0, 1]^c$ as the predicted probabilities assigned to each class. Then the loss can be expressed as

$$\mathcal{L}(\mathbf{Y}, \hat{\mathbf{Y}}) = -\frac{1}{N} \sum_j \left[\text{sum}(\mathbf{y}_j \cdot \log \hat{\mathbf{y}}_j) \right]$$

where sum denotes the summation over all elements of the vector, log and $\cdot$ are all element-wise.

After doing the derivations, we obtain the gradient with respect to W and $\mathbf{b}$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W} &= \frac{1}{N} \sum_i \left[\mathbf{x}_i^\top (\hat{\mathbf{y}}_i - \mathbf{y}_i) \right] \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}} &= \frac{1}{N} \sum_i \left[\hat{\mathbf{y}}_i - \mathbf{y}_i \right] \end{aligned}$$

Verify the correctness of this gradient in your own time! :), or check out this [great tutorial](#) for the derivation. Note that because W is a (d, c) matrix, $\frac{\partial \mathcal{L}}{\partial W}$ too. $\mathbf{x}_i^\top (\hat{\mathbf{y}}_i - \mathbf{y}_i)$ is therefore the **outer product** between the error vector $\hat{\mathbf{y}}_i - \mathbf{y}_i$ (c elements) and the input vector $\mathbf{x}$ (d elements).

For more efficiency, we can write the above expression in matrix form:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W} &= \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{Y}} - \mathbf{Y}) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}} &= \frac{1}{N} (\hat{\mathbf{Y}} - \mathbf{Y}) \end{aligned}$$

Now, let's put all these together and implement a function for the gradient and loss calculation:

TODOs: Read and complete the missing lines in `gradient_loss` function of the `SentimentClassifier` class in `gradient_descent.py` to calculate the gradients of the weights and biases of the linear layer, as well as the cross entropy loss.

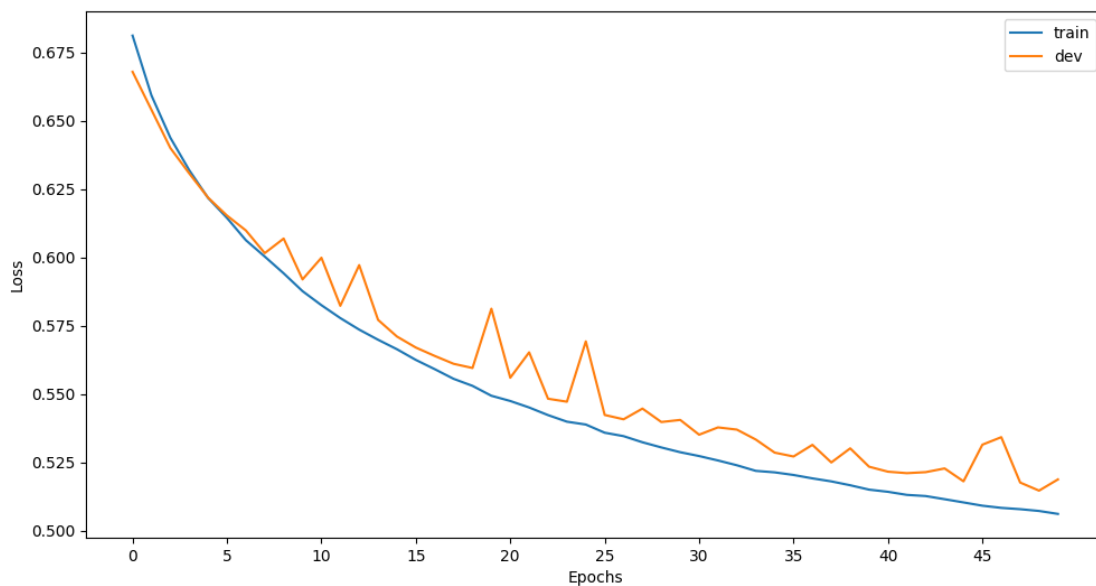
Hint: refer to [\[eq:grad_w\]](#) and [\[eq:grad_b\]](#) for the gradient calculation formulas, and [\[eq:ce\]](#) for cross-entropy loss calculation. Also, note that the weight matrix that is stored in `nn.Linear` is actually in the shape of $c \times d$, so you probably need to take a transpose of your gradient calculation.

Gradient Descent

Remember the gradient descent calculation we learned in class, with the gradient and loss calculation we just implemented, we can now optimize our classifier with gradient descent.

TODOs Read and complete the missing lines in `train` function in `gradient_descent.py`, to perform gradient updates on weights and biases, with the given `learning_rate`. Once you finish, run the `single_run_back_prop` in `main.py` to train the model and paste the plot here.

Hint: you can access the weights and biases of the linear layer with `model.linear.weight` and `model.linear.bias`, respectively.



Optional Feedback

Have feedback for this assignment? Found something confusing? We'd love to hear from you!